

Simulador de Concurrencia: Evidencia Experimental de Condiciones de Carrera y Comparación de Cuatro Mecanismos de Sincronización en C++17

Isaac Felipe Morún Moreira

Universidad Invenio — Licenciatura en Tecnologías de Información y Comunicación Empresarial

TIIT2007 – Sistemas Operativos | Agosto 2026

1. Resumen

Este documento presenta el diseño, la implementación y la evaluación experimental de un simulador del problema Productor-Consumidor desarrollado en C++17 con hilos reales, cuyo propósito es demostrar de forma medible el efecto de la sincronización sobre la integridad de los datos compartidos. Se implementaron cuatro versiones del mismo buffer circular de tamaño finito que difieren únicamente en su mecanismo de coordinación: sin sincronización, con exclusión mutua mediante mutex y espera activa, con mutex y variable de condición, y con tres semáforos construidos sobre las primitivas anteriores. La versión sin sincronización es deliberadamente vulnerable y su corrupción fue certificada de forma independiente con ThreadSanitizer, que reportó cinco condiciones de carrera distintas señalando archivo y línea; las tres versiones sincronizadas no produjeron ningún hallazgo. Sobre cinco corridas de cada versión con cuatro productores, cuatro consumidores, buffer de ocho posiciones y cincuenta mil ítems, la versión sin sincronización nunca entregó correctamente más del 80 % de los ítems producidos, y en la serie detallada que se reporta se situó entre el 34 % y el 42 %, mientras que las tres versiones sincronizadas entregaron el 100 % en todas las corridas sin excepción. El costo medido de la corrección fue de aproximadamente un orden de magnitud por ítem entregado. Un barrido de capacidad reveló que la conveniencia relativa entre espera activa y bloqueo se invierte según la contención, y un experimento de control en una segunda plataforma demostró que la variable causal de esa inversión es la sobresuscripción de núcleos y no el sistema operativo. Se incluye además una demostración de interbloqueo que reproduce las cuatro condiciones de Coffman y dos técnicas de prevención.

Palabras clave—concurrencia, condición de carrera, sección crítica, exclusión mutua, mutex, variable de condición, semáforo, interbloqueo, C++17.

2. Introducción

2.1 Contexto del problema

Cuando dos o más hilos de ejecución acceden concurrentemente a una misma estructura de datos y al menos

uno de ellos la modifica, el resultado del programa puede depender del orden en que el planificador del sistema operativo intercale sus instrucciones. Esa dependencia del orden se denomina condición de carrera, y constituye una de las fuentes de error más difíciles de diagnosticar en el software de sistemas, precisamente porque no se manifiesta de forma determinista: el mismo binario, con la misma entrada, puede producir un resultado correcto en una ejecución e incorrecto en la siguiente.

El problema Productor-Consumidor es el caso canónico para estudiar este fenómeno. Un conjunto de hilos productores genera elementos y los deposita en un buffer compartido de capacidad finita, mientras un conjunto de hilos consumidores los extrae. La coordinación necesaria no es trivial: los productores deben esperar si el buffer está lleno, los consumidores deben esperar si está vacío, y ambos grupos deben evitar interferir entre sí al manipular los índices y el contador del buffer.

La dificultad pedagógica de este tema es que la ausencia de sincronización no siempre produce fallos visibles. Un programa incorrecto puede funcionar durante miles de ejecuciones antes de manifestar el error. Por ello, el objetivo de este trabajo no es solamente implementar una solución correcta, sino construir una infraestructura de medición que permita cuantificar la diferencia entre una implementación vulnerable y una correcta, y que sostenga esa cuantificación con evidencia reproducible.

2.2 Objetivo general

Implementar en C++17 un simulador del problema Productor-Consumidor con hilos reales que permita comparar cuantitativamente, sobre la misma carga de trabajo y la misma estructura de datos, una versión deliberadamente carente de sincronización frente a versiones correctamente sincronizadas, documentando tanto la corrupción producida como el costo de evitarla.

2.3 Objetivos específicos

- Diseñar un buffer compartido circular de tamaño finito cuya sección crítica quede explícitamente identificada en comentarios.

- Implementar una versión sin sincronización que sea honestamente vulnerable, sin esperas artificiales introducidas para forzar el fallo.
- Diseñar un esquema de detección de corrupción que no introduzca por sí mismo condiciones de carrera en la medición.
- Implementar exclusión mutua correcta con `std::mutex` y verificar la ausencia total de inconsistencias.
- Implementar una variante con `std::condition_variable` que elimine la espera activa, y una variante con semáforos contruidos a mano.
- Cuantificar la diferencia entre versiones mediante un mínimo de cinco corridas de cada una, registrando ítems perdidos, duplicados, fantasma y corruptos.
- Analizar el sobrecosto introducido por la sincronización, normalizado por ítem entregado correctamente.
- Obtener evidencia independiente de la existencia de las condiciones de carrera mediante un analizador dinámico externo.
- Verificar la compilación y ejecución en una segunda plataforma y una segunda versión mayor del compilador.
- Demostrar experimentalmente un interbloqueo y las técnicas de prevención asociadas.

2.4 Alcance y limitaciones

El trabajo modela exclusivamente el patrón Productor-Consumidor sobre un buffer circular en memoria compartida, con hilos del mismo proceso. Los ítems transportados son estructuras sintéticas diseñadas para permitir la detección de corrupción; no representan una carga de trabajo aplicativa real y el simulador no realiza operaciones de entrada/salida ni cómputo dentro de la sección crítica.

Quedan explícitamente fuera del alcance otros dos casos clásicos de la unidad. El problema de Lectores-Escritores no se implementa porque responde a un patrón de acceso distinto —múltiples lectores simultáneos frente a escritores exclusivos— que el enunciado del proyecto no solicita. El problema de los Filósofos Comensales no se implementa en su forma completa de cinco filósofos, pero su mecanismo esencial, la espera circular sobre recursos compartidos, sí se reproduce y se analiza en la demostración de interbloqueo de la Sección 6.7, reducido a dos comensales y dos tenedores. El algoritmo del banquero, correspondiente a la estrategia de evitación de interbloqueos, se omite deliberadamente por pertenecer al ámbito de asignación de múltiples tipos de recursos que corresponde al cuarto proyecto del curso.

Se declara asimismo que la variante basada en semáforos excede lo requerido por el enunciado, que solicita únicamente exclusión mutua con `mutex` y menciona las variables de condición como opcionales. Se incorporó por su valor comparativo y porque permite contrastar experimentalmente la distinción entre propiedad y ausencia de propiedad discutida en la Sección 3.3.

3. Marco Teórico

3.1 Condición de carrera y sección crítica

Se denomina sección crítica al fragmento de código que accede a un recurso compartido y que, para preservar la consistencia de ese recurso, debe ejecutarse sin intercalación con otras secciones críticas sobre el mismo recurso [1]. Una condición de carrera ocurre cuando el resultado del programa depende del orden relativo en que varios hilos ejecutan sus secciones críticas.

El caso elemental es el incremento de un contador compartido. La expresión que aparenta ser una sola operación se traduce en tres pasos a nivel de máquina: leer el valor actual, sumarle uno y escribir el resultado. Si dos hilos ejecutan esos tres pasos de forma intercalada, ambos pueden leer el mismo valor inicial, ambos calcular el mismo resultado y ambos escribirlo, de modo que uno de los dos incrementos se pierde sin dejar rastro.

Conviene distinguir dos conceptos que suelen confundirse. Una carrera de datos, en la terminología del estándar del lenguaje, es la situación en que dos hilos acceden a la misma posición de memoria sin un orden establecido entre ellos y al menos uno de los accesos es de escritura; en C++ esto constituye comportamiento indefinido. Una condición de carrera, en la terminología de sistemas operativos, es la propiedad de que el resultado observable dependa del orden de ejecución. Toda carrera de datos puede provocar una condición de carrera, pero existen condiciones de carrera sin carrera de datos, por ejemplo cuando dos operaciones individualmente atómicas se ejecutan en un orden no previsto.

La distinción tiene una consecuencia práctica relevante para este trabajo: dado que la versión sin sincronización incurre formalmente en comportamiento indefinido, lo que se observa al ejecutarla no es «el» comportamiento del programa sino la manifestación concreta que produce un compilador determinado sobre una arquitectura determinada. Esta consideración se retoma en la Sección 8.2.

3.2 Exclusión mutua y `mutex`

La exclusión mutua es el requisito de que a lo sumo un hilo se encuentre dentro de la sección crítica en cada instante. Una solución correcta debe satisfacer además dos propiedades adicionales [1]: progreso, es decir que si ningún hilo está

dentro de la sección crítica alguno de los que desean entrar debe poder hacerlo; y espera acotada, es decir que ningún hilo espere indefinidamente mientras otros entran repetidamente.

El mutex es el mecanismo que implementa directamente la exclusión mutua. Su propiedad distintiva es la pertenencia: el hilo que adquiere el cerrojo es el único autorizado a liberarlo. Esa restricción, que puede parecer una limitación, es lo que permite razonar localmente sobre la corrección del código protegido y lo que habilita optimizaciones del sistema operativo, como la herencia de prioridad.

3.3 Espera bloqueante: variables de condición y semáforos

Un mutex resuelve la exclusión mutua pero no la espera por una condición. Si un productor encuentra el buffer lleno, necesita esperar a que un consumidor libere espacio. La forma más simple de esperar es consultar la condición en un bucle —espera activa— pero ese enfoque consume procesador mientras no hay trabajo que hacer.

La variable de condición permite que el hilo se bloquee liberando el mutex de forma atómica y sea despertado cuando otro hilo notifique el cambio. Su uso correcto exige reevaluar la condición al despertar, porque una variable de condición puede producir despertares espurios, es decir retornos de la espera sin que ninguna notificación se haya emitido.

El semáforo, propuesto por Dijkstra, es un contador entero con dos operaciones atómicas: P o wait, que espera a que el contador sea positivo y lo decrementa, y V o signal, que lo incrementa y despierta a un eventual hilo en espera [2]. A diferencia del mutex, el semáforo no tiene propietario: cualquier hilo puede señalarlo, no solamente el que esperó. Esa ausencia de pertenencia es justamente lo que lo hace apropiado para la sincronización entre hilos de roles distintos, como se explota en la Sección 5.3.

3.4 Interbloqueo y condiciones de Coffman

Un interbloqueo es la situación en que dos o más hilos quedan detenidos de forma permanente porque cada uno espera un recurso retenido por otro. Coffman estableció que un interbloqueo requiere la presencia simultánea de cuatro condiciones [1], [2]: exclusión mutua, es decir que al menos un recurso sea no compartible; retención y espera, es decir que un hilo conserve recursos mientras solicita otros; ausencia de apropiación, es decir que los recursos no puedan retirarse por la fuerza; y espera circular, es decir que exista una cadena cerrada de hilos esperándose mutuamente.

La utilidad práctica de esta caracterización es que basta con impedir una sola de las cuatro condiciones para que el interbloqueo sea imposible por construcción. Las estrategias de manejo se clasifican en prevención, que elimina estructuralmente alguna condición; evitación, que analiza el estado antes de conceder cada recurso; y detección con

recuperación, que permite que el interbloqueo ocurra y lo descubre observando la ausencia de progreso [1].

3.5 Relación con la observación práctica en xv6

El kernel didáctico xv6, utilizado como referencia práctica del curso, protege sus estructuras compartidas mediante spinlocks, implementados en spinlock.c con las funciones acquire() y release(). Un spinlock es exactamente una espera activa: el procesador que no logra adquirir el cerrojo permanece girando en un bucle hasta conseguirlo, en lugar de ceder el procesador.

La pregunta natural es por qué un kernel elegiría desperdiciar procesador de esa manera en lugar de bloquear el hilo. La respuesta que suele darse es que las secciones críticas del kernel son muy breves y que el costo de un cambio de contexto superaría al de girar unas pocas iteraciones. Este trabajo permite contrastar esa afirmación con datos propios, porque la variante B es precisamente el análogo en espacio de usuario de un spinlock y la variante C su contraparte bloqueante; el resultado se presenta en la Sección 6.6 y se discute en la Sección 8.2.

4. Diseño de la Solución

4.1 Arquitectura general

El simulador se organiza en cuatro capas con responsabilidades disjuntas. La capa de datos define el ítem transportado y el tipo de reloj empleado. La capa de buffers contiene las cuatro implementaciones intercambiables del buffer compartido, cada una en su propio encabezado. La capa de instrumentación define los registros privados por hilo y la función de reconciliación posterior. La capa de orquestación crea los hilos, mide el tiempo y emite los resultados.

Una decisión de diseño relevante es que las cuatro variantes de buffer no derivan de una clase base con métodos virtuales, sino que son clases independientes seleccionadas mediante una función plantilla en el orquestador. El motivo es de medición: una llamada virtual en el camino de ejecución más frecuente introduciría una indirección que se sumaría al tiempo medido y contaminaría precisamente la magnitud que el proyecto se propone cuantificar.

4.2 Estructura del ítem y del buffer

El ítem transportado consta de cuatro campos: identificador del productor, número de secuencia dentro de ese productor, identificador único global derivado de los dos anteriores, y una suma de verificación calculada a partir de los tres primeros.

La elección de cuatro campos en lugar de un único entero es deliberada y responde a una propiedad del hardware. En la arquitectura x86-64 una escritura alineada de ocho bytes es

atómica a nivel de instrucción, de modo que si el ítem fuese un solo entero nunca podría observarse un valor parcialmente escrito: la corrupción se limitaría a pérdidas y sobrescrituras. Con cuatro campos, en cambio, dos productores pueden intercalar sus escrituras campo por campo y dejar en el buffer un ítem cuyos campos provienen de dos escrituras distintas, es decir, un ítem que ningún productor generó. La suma de verificación permite detectar exactamente esa situación, dado que un ítem así no satisface la relación entre sus campos.

La suma de verificación incorpora una constante no nula, de modo que una posición del buffer que nunca fue escrita —cuyos campos valen cero— no supere la verificación y sea distinguible de un ítem legítimo. Por la misma razón los identificadores de productor comienzan en uno y no en cero, garantizando que el identificador único global nunca valga cero.

El buffer es un arreglo circular de capacidad fija con tres variables de estado: el índice de escritura, el índice de lectura y el contador de elementos presentes. El contador es la variable más frágil del diseño, dado que su actualización es la operación de lectura-modificación-escritura descrita en la Sección 3.1. Los índices se emplean siempre en aritmética modular respecto de la capacidad; conviene subrayar que esa aritmética es la propia de un buffer circular y no un mecanismo defensivo añadido, aunque tiene el efecto colateral de que un índice corrompido produzca sobrescritura en lugar de acceso fuera de rango.

4.3 Esquema de detección de corrupción

El diseño de la instrumentación está sujeto a una restricción que condiciona la validez de todos los resultados: si los hilos registraran lo producido y lo consumido en una estructura compartida, ese registro sufriría a su vez condiciones de carrera y las cifras obtenidas no probarían nada. La regla adoptada es que durante la ejecución ningún hilo comparte estructura de medición con otro.

En consecuencia, cada productor acumula en un vector propio los identificadores que entregó, y cada consumidor acumula en un vector propio los ítems que extrajo. La reconciliación se realiza después de unir todos los hilos, en un único hilo de ejecución. De este modo el fenómeno medido es no determinista pero la medición es determinista y auditable. Los vectores se reservan con su capacidad final antes de iniciar la medición, para que ninguna reasignación de memoria se compute dentro del intervalo cronometrado.

Tabla 1: Métricas obtenidas en la reconciliación posterior a la unión de hilos.

Métrica	Definición operativa
Producidos	Suma de los tamaños de los vectores de los productores
Consumidos	Suma de los tamaños de los vectores de los consumidores

Métrica	Definición operativa
Perdidos	Identificadores producidos que no aparecen en ningún consumidor
Duplicados	Identificadores consumidos más de una vez
Fantasma	Identificadores consumidos que ningún productor generó
Corruptos	Ítems cuya suma de verificación no se satisface
Entregados OK	Ítems válidos entregados exactamente una vez
Violación de capacidad	Máximo del contador observado por encima de la capacidad

Las tres últimas métricas satisfacen una identidad que se verifica en cada corrida y que sirve como control de consistencia interna de la instrumentación: los ítems entregados correctamente equivalen tanto a los producidos menos los perdidos como a los consumidos menos duplicados, fantasma y corruptos.

4.4 Supuestos de diseño

Terminación por cuota fija. Cada consumidor conoce de antemano cuántos ítems debe extraer, calculado como el total dividido entre el número de consumidores. La alternativa —que los hilos consulten un estado compartido para decidir cuándo detenerse— haría que la terminación dependiera del contador, que en la versión sin sincronización puede corromperse; el programa podría entonces no terminar nunca.

Doble límite temporal. Se emplean dos límites de naturaleza distinta. El vigilante corta la corrida completa y, si se activa, la corrida se marca como no válida para el informe. El corte por falta de progreso determina cuánto tolera un hilo sin avanzar antes de rendirse, y es necesario porque la versión sin sincronización pierde ítems y sus consumidores nunca alcanzarían su cuota. Debe subrayarse que ninguno de los dos límites crea ni evita la corrupción: únicamente deciden cuándo dejar de esperar algo que no va a llegar. En las versiones sincronizadas no se pierde ningún ítem y el corte por falta de progreso nunca llega a activarse.

Medición hasta la última operación exitosa. El tiempo no se mide hasta que los hilos terminan sino hasta el instante de la última operación exitosa registrada por cualquiera de ellos. De otro modo, el tiempo muerto de los consumidores que esperan ítems que ya se perdieron dominaría la medición de la versión sin sincronización y las variantes serían incomparables.

Flags de compilación idénticos. Las cuatro variantes se compilan con exactamente las mismas banderas, de modo que cualquier diferencia observada sea atribuible al mecanismo de sincronización y no al tratamiento del compilador.

5. Implementación

5.1 Lenguaje y estándar

El proyecto está escrito en C++17 y utiliza exclusivamente la biblioteca estándar, sin dependencias externas. La elección de

C++17 en lugar de C++20 es deliberada: se verificó que la plantilla `std::counting_semaphore`, que habría simplificado la variante D, requiere C++20 y no compila bajo C++17. Se optó por construir el semáforo manualmente para conservar el mismo estándar que el proyecto anterior del curso y reducir el riesgo en la verificación multiplataforma, con el beneficio adicional de que la implementación explícita evidencia la comprensión del mecanismo.

5.2 Módulos

Tabla 2: Módulos del proyecto y su responsabilidad.

Archivo	Responsabilidad
Item.h	Estructura del ítem, suma de verificación y tipo de reloj
BufferA.h	Buffer sin sincronización
BufferB.h	Buffer con mutex y espera activa
BufferC.h	Buffer con mutex y variable de condición
BufferD.h	Buffer con tres semáforos
Semaforo.h	Semáforo contador construido a mano
Registro.h/.cpp	Registros privados por hilo y reconciliación
main.cpp	Orquestación, argumentos y salida
reporte.cpp	Generación de tablas y gráficos desde los CSV
deadlock.cpp	Demostración de interbloqueo y prevención

5.3 Las cuatro variantes

Las cuatro implementaciones comparten la misma interfaz y la misma estructura de datos; difieren únicamente en cómo protegen la sección crítica y en cómo esperan cuando la operación no puede completarse.

Tabla 3: Variantes implementadas y comparación que aísla cada una.

Var.	Mecanismo	Tipo de espera	Comparación que aísla
A	Ninguno	Activa	Referencia: la corrupción
B	<code>std::mutex</code>	Activa	B frente a A: costo del mutex
C	<code>mutex + condition_variable</code>	Bloqueante	C frente a B: efecto de no girar
D	Tres semáforos	Bloqueante	D frente a C: arquitectura del mecanismo

El propósito de mantener la misma estrategia de espera entre A y B es metodológico: si la versión sincronizada se comparara directamente contra la versión que emplea variable de condición, el número resultante mezclaría el costo de bloquear con el efecto de dejar de consumir procesador, y no sería interpretable.

En la variante B la diferencia esencial respecto de A cabe en una línea: la comprobación de que hay espacio disponible y la escritura correspondiente ocurren dentro del mismo bloqueo. El cerrojo se libera antes de reintentar; de no hacerlo, un productor esperaría a que se libere espacio mientras impide que cualquier consumidor lo libere, produciendo un interbloqueo con las cuatro condiciones de Coffman.

```
for (;;) {
{
```

```
std::lock_guard<std::mutex> cerr(mutex_);
if (cuenta_ < capacidad_) {
    datos_[cabeza_] = it;
    cabeza_ = (cabeza_ + 1) % capacidad_;
    cuenta_ = cuenta_ + 1;
    return true;
}
} // el cerrojo se libera aqui
if (Reloj::now() > limite) return false;
}
```

La variante C sustituye el bucle de reintento por una espera sobre variable de condición con predicado, forma que gestiona automáticamente los despertares espurios. Emplea dos variables de condición y no una: los productores esperan la condición «hay espacio» y los consumidores «hay datos», que son condiciones distintas; con una sola variable, cada notificación tendría probabilidad apreciable de despertar al tipo de hilo equivocado, que reevaluaría su predicado y volvería a dormirse. La notificación se emite tras liberar el cerrojo, para que el hilo despertado no lo encuentre ocupado.

La variante D emplea tres semáforos: uno que cuenta espacios libres, otro que cuenta ítems disponibles y un tercero binario que actúa como cerrojo. El cambio conceptual respecto de B y C es que la condición de lleno o vacío deja de evaluarse explícitamente y queda codificada en el contador del semáforo. Dos detalles de la implementación merecen mención porque constituyen errores frecuentes: el orden de adquisición debe tomar primero el semáforo de recurso y después el de exclusión, ya que el orden inverso produce interbloqueo; y si la segunda adquisición fracasa por tiempo, el permiso ya obtenido debe devolverse, pues de lo contrario el buffer pierde capacidad efectiva en cada corte hasta bloquearse por completo.

5.4 Compilación y ejecución

```
make # simulador, deadlock, reporte
make experimentos # 5 corridas x 4 variantes
make race # evidencia ThreadSanitizer
make deadlock # los tres modos

./simulador --variante A|B|C|D
[--productores N] [--consumidores N]
[--capacidad N] [--items N]
[--csv archivo] [--sin-color]
```

El generador de reportes se ejecuta sobre los archivos CSV producidos por el simulador, de modo que cualquier tabla del presente documento puede regenerarse sin volver a ejecutar los experimentos, y ninguna cifra fue transcrita manualmente desde la consola.

6. Resultados Experimentales

6.1 Entorno de prueba

Tabla 4: Entornos de ejecución empleados.

	Entorno 1	Entorno 2
Sistema	Ubuntu 26.04 LTS sobre VirtualBox	Windows 11 Pro (nativo)

	Entorno 1	Entorno 2
Compilador	g++ 15.2.0	g++ 16.2.0 (MinGW-w64 UCRT)
Núcleos disponibles	6 vCPU	16
Modelo de hilos	posix (nativo)	posix (winthreads)
Banderas	-std=c++17 -Wall -Wextra -pthread -O2	idénticas

Salvo indicación contraria, la configuración de los experimentos es de cuatro productores, cuatro consumidores, buffer de ocho posiciones y cincuenta mil ítems por corrida. Todas las cifras provienen de ejecuciones reales cuyos archivos CSV están versionados en el directorio results del repositorio.

6.2 Evidencia de condición de carrera

La Tabla 5 presenta el detalle de cinco corridas consecutivas de la versión sin sincronización. Se reproduce el desglose completo porque el enunciado exige evidencia explícita de al menos una condición de carrera capturada.

Tabla 5: Cinco corridas de la versión A. Columnas: perdidos, duplicados, corruptos, inconsistencias totales, entregados OK, porcentaje OK y segundos. 50 000 ítems por corrida.

#	Perd.	Dupl.	Corr.	Incons.	OK	%	s
1	30 107	8 762	634	39 503	19 893	39.8	0.00522
2	30 736	11 457	577	42 770	19 264	38.5	0.00441
3	29 229	5 667	1 083	35 979	20 771	41.5	0.00488
4	28 982	11 773	671	41 426	21 018	42.0	0.00580
5	32 859	5 917	869	39 645	17 141	34.3	0.00467
Media	30 383	8 715	767	39 865	19 617	39.2	0.00500

Tres observaciones sobre estos datos. La primera es la magnitud: menos del 40 % de los ítems producidos llega correctamente a un consumidor. La segunda es la presencia simultánea de los tres tipos de corrupción previstos en el diseño, incluidos los ítems corruptos, que constituyen la evidencia de escrituras parcialmente intercaladas y no meramente de errores de conteo.

La tercera y más relevante es la variabilidad. Los ítems duplicados oscilan entre 5 667 y 11 773 entre corridas del mismo binario sobre la misma entrada, una diferencia superior al doble. Esa dispersión es precisamente lo que acredita que la corrupción es genuina: un fallo fabricado mediante esperas artificiales o mediante impresión directa de mensajes de error produciría valores idénticos en todas las corridas.

La serie se repitió en una sesión posterior, tras una recompilación completa desde cero y con la máquina en reposo, obteniéndose entre 41 032 y 49 643 inconsistencias y una proporción de entrega correcta del 46 %. El fenómeno se reproduce de forma consistente en su naturaleza y su orden de magnitud, pero no en sus valores exactos; la variación entre sesiones de medición se analiza en la Sección 8.3.

6.3 Verificación independiente con ThreadSanitizer

Para sostener la afirmación de que las condiciones de carrera son reales y no artefactos del diseño experimental, se recompiló el simulador con el analizador dinámico ThreadSanitizer, integrado en GCC. La herramienta instrumenta los accesos a memoria y reporta las carreras detectadas indicando archivo, línea y los dos accesos en conflicto.

La versión A produjo cinco reportes de carrera distintos. El principal señala como lectura la línea correspondiente a la comprobación de que el buffer no está lleno y como escritura previa la línea que incrementa el contador, es decir, exactamente el patrón de comprobar y actuar descrito en la Sección 3.1.

```
WARNING: ThreadSanitizer: data race
Read of size 4 by thread T2:
  #0 BufferA::producir(..) BufferA.h:49

Previous write of size 4 by thread T1:
  #0 BufferA::producir(..) BufferA.h:81
```

Las variantes B, C y D, ejecutadas bajo el mismo analizador y completando la totalidad de sus ítems, no produjeron ningún reporte. Debe señalarse que el analizador introduce una penalización de rendimiento de entre cinco y quince veces, por lo cual los binarios instrumentados se emplearon exclusivamente para obtener esta evidencia y ninguna medición de tiempo del presente documento proviene de ellos.

6.4 Corrección de las variantes sincronizadas

Tabla 6: Corrección de las cuatro variantes. Cinco corridas de cada una.

Variante	Inconsistencias	% entregado OK	Carreras detectadas
A — sin sincronización	35 979 – 42 770	34.3 – 42.0	5
B — mutex, espera activa	0	100	0
C — mutex + cond. variable	0	100	0
D — semáforos	0	100	0

Las tres variantes sincronizadas produjeron cero inconsistencias en la totalidad de sus corridas, sin una sola excepción, y el máximo del contador observado nunca superó la capacidad del buffer. El contraste es cualitativamente distinto del habitual en comparaciones de rendimiento: no se trata de que una alternativa sea mejor por un margen, sino de que un comportamiento no determinista se vuelve completamente determinista.

6.5 Sobre costo de la sincronización

El análisis del sobre costo requiere una precaución metodológica. Comparar el tiempo total de la versión A contra el de una versión sincronizada sería incorrecto por dos motivos independientes: la versión A realiza menos trabajo efectivo, puesto que pierde el 60 % de los ítems, y consume procesador en espera activa mientras las variantes

bloqueantes lo ceden. Por ello la magnitud reportada es el tiempo por ítem entregado correctamente.

Se observó además que los tiempos absolutos derivan considerablemente entre sesiones de medición en el entorno virtualizado; se llegó a registrar un factor de diez con el mismo binario sin recompilar. La causa es que la máquina virtual comparte los núcleos físicos con el sistema anfitrión y la espera activa es extremadamente sensible a la disponibilidad real de procesador. En consecuencia, todas las mediciones comparativas se realizan intercalando las variantes dentro de una misma tanda, de modo que todas experimenten condiciones equivalentes, y se reportan cocientes y medianas en lugar de tiempos absolutos aislados.

Tabla 7: Costo por ítem entregado correctamente, buffer de 8 posiciones. Medianas de tandas intercaladas.

Variante	Entorno 1 (µs)	Entorno 2 (µs)
A — sin sincronización	2.02	0.30
B — mutex, espera activa	30.44	0.90
C — mutex + cond. variable	125.22	2.83
D — semáforos	124.58	6.07

En el entorno virtualizado, el paso de la versión sin sincronización a la versión con mutex y la misma estrategia de espera representó un factor comprendido entre 9.8 y 20.8 según la tanda, con mediana de 17.4. Ese es el sobre costo atribuible al mutex en condiciones de contención elevada.

6.6 Efecto de la contención sobre la estrategia de espera

Un resultado inicialmente contraintuitivo motivó un experimento adicional. Con buffer de ocho posiciones, la variante que se duerme resultó considerablemente más lenta que la que gira, en contra de la intuición de que la espera activa desperdicia recursos. Para determinar si el efecto dependía del grado de contención se realizó un barrido de capacidad.

Tabla 8: Barrido de capacidad en el Entorno 1. Costo por ítem en µs, medianas de tres corridas intercaladas.

Capacidad	A	B	C	D	C/B
8	2.08	30.18	118.11	101.42	3.91
64	0.92	16.51	44.26	41.89	2.68
512	2.22	11.74	10.33	11.23	0.88
4 096	1.94	2.55	1.80	3.11	0.71
32 768	0.44	2.55	1.37	2.99	0.54

El cociente entre ambas estrategias recorre un rango amplio y cambia de signo entre las capacidades 512 y 4 096. Con buffer pequeño la variante bloqueante cuesta casi cuatro veces más; con buffer grande cuesta aproximadamente la mitad. La interpretación se desarrolla en la Sección 8.2.

6.7 Demostración de interbloqueo

Se desarrolló un programa independiente que reproduce un interbloqueo real entre dos hilos y dos cerrojos, denominados en el código tenedor izquierdo y derecho por analogía con el

problema de los Filósofos Comensales reducido a dos comensales. El primer hilo adquiere los cerrojos en un orden y el segundo en el orden inverso. No se emplean esperas artificiales: la operación simplemente se repite hasta que el planificador produce la intercalación que provoca el bloqueo.

Dado que un hilo interbloqueado no puede ser unido nunca, los hilos se desacoplan y se vigilan externamente observando si el contador de trabajo progresa. Ese mecanismo constituye, en sí mismo, una implementación de la estrategia de detección descrita en la Sección 3.4.

Tabla 9: Operaciones completadas antes del interbloqueo, sobre 200 000 previstas.

Corrida	Entorno 1	Entorno 2
1	30 057	93
2	1 929	107
3	35 904	342
4	1 738	32
5	6 988	139

Las cinco corridas se bloquearon en ambos entornos, pero en puntos muy dispares: en el Entorno 1 el rango abarca un factor de veinte entre el mínimo y el máximo. Al igual que con la condición de carrera, esa dispersión acredita que el fenómeno es genuino y no está guionado.

Se implementaron dos técnicas de prevención, cada una dirigida a una condición de Coffman distinta. La primera consiste en que ambos hilos adquieran los cerrojos en el mismo orden, lo que elimina la espera circular. La segunda emplea una adquisición conjunta de ambos cerrojos como operación única, lo que elimina la retención y espera. Ambas completaron las 200 000 operaciones en todas las corridas. Sobre cinco repeticiones de cada técnica en el Entorno 1 las medianas fueron 0.0561 s y 0.0564 s respectivamente, con valores individuales comprendidos entre 0.051 s y 0.136 s en ambos modos. La dispersión de la medición supera ampliamente la diferencia entre técnicas, de modo que lo que puede afirmarse es que ninguna resultó apreciablemente más costosa que la otra, en marcado contraste con el sobre costo de la exclusión mutua reportado en la Sección 6.5.

6.8 Verificación multiplataforma

El código compiló sin modificaciones y sin una sola advertencia bajo -Wall -Wextra en ambos entornos, que difieren no solamente en el sistema operativo sino también en la versión mayor del compilador. Se resolvió además una duda de portabilidad planteada al inicio del proyecto: si la bandera -pthread, necesaria en Linux, lo es también en MinGW.

La verificación consistió en compilar con y sin la bandera y comparar los resultados. Ambos ejecutables presentan idéntico tamaño y lista de bibliotecas dinámicas, incluida la biblioteca de hilos POSIX, que se enlaza en ambos casos. Los binarios difieren en un byte del encabezado ejecutable, pero

dos compilaciones realizadas con banderas idénticas difieren en el mismo byte, lo que identifica esa diferencia como la marca temporal de compilación y no como un efecto de la bandera. Se concluye que la bandera es necesaria en Linux e inocua en MinGW-w64 con modelo de hilos POSIX, y se conserva en el archivo de compilación por portabilidad.

Tabla 10: Comparación de corrupción entre entornos, versión A, buffer de 8.

Métrica	Entorno 1	Entorno 2
Inconsistencias, tanda A	35 979 – 42 770	56 356 – 59 805
Inconsistencias, tanda B	19 378 – 25 521	no medida
% entregado OK, tanda A	34.3 – 42.0	38.0
% entregado OK, tanda B	73.0	no medida

Estos datos deben leerse con cautela y se comentan en la Sección 8.3 como limitación. El conteo de inconsistencias fue superior en el Entorno 2 que en cualquier tanda del Entorno 1, lo cual resulta compatible con la hipótesis de que un mayor número de núcleos incrementa el paralelismo efectivo y con ello la frecuencia de intercalaciones perjudiciales. Sin embargo, la proporción de ítems entregados correctamente en el Entorno 2 se sitúa dentro del rango observado entre distintas tandas del Entorno 1, de modo que la variabilidad entre sesiones de medición es del mismo orden que la diferencia atribuida a la plataforma. Los datos recogidos no permiten, por tanto, afirmar un efecto de plataforma sobre la tasa de corrupción.

Sí resultó concluyente, en cambio, un experimento de control sobre la estrategia de espera. En el Entorno 2, con la misma configuración de ocho hilos, el cruce observado en la Tabla 8 no se reprodujo: la variante que gira resultó igual o superior en casi todas las capacidades. La diferencia entre ambos entornos no era únicamente el sistema operativo, sino la relación entre hilos y núcleos: ocho hilos sobre seis procesadores virtuales en el primer caso, ocho sobre dieciséis en el segundo. Se repitió entonces la medición en el Entorno 2 elevando la carga a treinta y dos hilos para saturar los dieciséis núcleos.

Tabla 11: Experimento de control en el Entorno 2 con 32 hilos sobre 16 núcleos.

Capacidad	B (μs)	C (μs)	C/B
512	1.10	0.65	0.59
4 096	0.95	0.51	0.54
32 768	0.48	0.47	0.99

Al saturar los núcleos, el cruce reaparece en el segundo entorno con valores prácticamente idénticos a los del primero. Se concluye que la variable causal de la inversión no es el sistema operativo sino la sobresuscripción de núcleos.

7. Evaluación ISO/IEC 25010

Autoevaluación conforme a la rúbrica institucional, que define cinco características de calidad para el ámbito del curso, en escala de Excelente (4) a Insuficiente (1). Cada calificación se justifica con evidencia verificable en el repositorio.

Tabla 12: Autoevaluación ISO/IEC 25010.

Característica	Cal.	Justificación
Rendimiento	4	Se mide y reporta el costo por ítem entregado correctamente para las cuatro variantes, con comparación entre escenarios: barrido de cinco capacidades, dos entornos y un experimento de control con carga elevada (Tablas 7, 8 y 11). La metodología de medición intercalada se adoptó tras detectar deriva de los tiempos absolutos, y se reportan medianas y rangos en lugar de valores aislados.
Fiabilidad	4	Las tres variantes sincronizadas produjeron cero inconsistencias en la totalidad de las corridas, resultado confirmado de forma independiente con ThreadSanitizer. El programa valida los argumentos de línea de comandos, rechaza configuraciones que incumplen el mínimo de dos productores y dos consumidores, y distingue el corte por vigilante del corte por falta de progreso, marcando como no válidas las corridas afectadas por el primero.
Usabilidad	3	La interfaz ofrece ayuda en línea, mensajes de error específicos por argumento, salida coloreada que resalta el contraste entre variantes y opciones para desactivar color y caracteres extendidos. La calificación no alcanza el máximo porque la interpretación correcta de las métricas de salida — en particular la distinción entre ítems fantasma y corruptos — requiere consultar la documentación; la salida no es autoexplicativa para un usuario que desconozca el diseño del detector.
Compatibilidad	4	Compila y ejecuta sin modificaciones ni advertencias en dos sistemas operativos y dos versiones mayores del compilador (Tabla 4). Se verificó experimentalmente el comportamiento de la bandera -pthread en ambas plataformas comparando binarios y dependencias. El generador de reportes ofrece un modo cuya salida se comprobó libre de caracteres fuera del rango ASCII, para consolas sin soporte de codificación extendida.
Mantenibilidad	4	Cada variante de sincronización reside en su propio encabezado con interfaz común, de modo que agregar una quinta no requiere modificar la orquestación ni la instrumentación. Las secciones críticas están delimitadas explícitamente en comentarios, según exige el enunciado. El archivo de compilación declara los encabezados como dependencia tras detectarse que su omisión provocaba medir binarios desactualizados.

La única calificación inferior al máximo corresponde a una limitación real y verificable, no a modestia retórica. Se ha evitado deliberadamente incorporar características que la rúbrica institucional no contempla para el ámbito del curso, así como afirmar cualquier propiedad que no cuente con un comando capaz de reproducirla.

8. Discusión de Resultados

8.1 Interpretación: qué mecanismo resultó superior y bajo qué condiciones

En el eje de la corrección la respuesta es categórica y no admite matices: las tres variantes sincronizadas son equivalentes entre sí y absolutamente superiores a la versión

sin sincronización. No existe compensación posible en la que un 40 % de entrega correcta resulte aceptable.

En el eje del rendimiento, en cambio, no existe un ganador único, y esa es la conclusión más útil del trabajo. Bajo contención elevada la espera activa es preferible por un margen amplio, alcanzando un factor cercano a cuatro respecto de la variante bloqueante. Bajo contención reducida y con sobresuscripción de núcleos la relación se invierte y la variante bloqueante resulta aproximadamente el doble de eficiente. La variante basada en semáforos se comporta como la bloqueante cuando la contención es alta, pero se degrada cuando es baja: cada operación atraviesa tres semáforos, y cada semáforo contiene internamente su propio mutex y su propia variable de condición, de modo que su costo fijo por operación es estructuralmente superior al de una única sección crítica.

Resulta ilustrativo que ese costo fijo se manifieste de forma distinta según la plataforma. Con buffer de ocho posiciones, las variantes C y D son indistinguibles en el Entorno 1 pero difieren en un factor de 2.1 en el Entorno 2. Puesto que el semáforo se construye sobre mutex y variable de condición, la diferencia debe atribuirse al costo relativo de esas primitivas en cada implementación de la biblioteca de hilos.

8.2 Relación con la teoría

Tres observaciones del experimento admiten explicación teórica directa.

La ubicación de la comprobación explica toda la corrupción. La diferencia entre la versión A y la B no reside en la cantidad de código ni en su complejidad, sino en si la comprobación de la condición y la acción correspondiente ocurren dentro del mismo bloqueo. El reporte de ThreadSanitizer confirma esta lectura de forma independiente al señalar como par en conflicto precisamente la línea de comprobación y la línea de incremento del contador. Es la manifestación empírica de que la sección crítica no es un conjunto de instrucciones peligrosas sino un intervalo que debe permanecer indivisible.

El cruce entre estrategias de espera confirma y precisa la justificación del spinlock. La explicación habitual de por qué un kernel emplea espera activa es que sus secciones críticas son breves. Los datos obtenidos sostienen esa explicación y añaden una precisión: la variable determinante no es únicamente la duración de la sección crítica sino la relación entre el tiempo esperado de espera y el costo de un cambio de contexto, mediada por la disponibilidad de procesadores. Con buffer pequeño la espera dura nanosegundos, porque otro hilo libera espacio casi de inmediato, y dormirse implica una llamada al sistema que cuesta microsegundos; girar resulta entonces varios órdenes de magnitud más barato. Con buffer grande la espera es infrecuente, pero cuando ocurre bajo sobresuscripción el hilo

que gira consume el procesador que necesita justamente aquel que podría desbloquearlo. El experimento de control de la Tabla 11 aísla esa variable de forma inequívoca.

El comportamiento indefinido no es una abstracción del estándar. La versión A incurre formalmente en comportamiento indefinido, y esto tuvo una consecuencia concreta durante el desarrollo. La comprobación del límite temporal se ubicó dentro del bucle de espera por dos razones, una de las cuales es que invoca una función externa que el compilador no puede analizar y que por tanto lo obliga a releer la variable compartida en cada iteración. Sin ella, el optimizador está autorizado a asumir que la variable no cambia y a convertir el bucle en una espera infinita. Lo observado en la versión A no es, por tanto, el comportamiento del programa sino su manifestación concreta bajo un compilador y una arquitectura determinados.

8.3 Limitaciones encontradas durante el desarrollo

La comparación de corrupción entre plataformas no es concluyente. Es la limitación más significativa. Los datos de la Tabla 10 muestran un conteo de inconsistencias superior en el segundo entorno, pero la proporción de ítems entregados correctamente en ese entorno cae dentro del rango observado entre distintas tandas del primero. Dicho de otro modo, la variabilidad entre sesiones de medición en una misma plataforma es del mismo orden que la diferencia atribuida al cambio de plataforma. A lo largo del desarrollo se registraron cuatro sesiones de medición en el primer entorno, con proporciones medias de entrega correcta del 39 %, 46 %, 73 % y 80 %; el valor del segundo entorno, 38 %, queda por debajo de todas ellas pero dentro del orden de la más baja. Establecer un efecto de plataforma requeriría un número considerablemente mayor de tandas por entorno y un control más estricto de la carga del sistema anfitrión. La afirmación se reporta como observación no concluyente.

Los tiempos absolutos del entorno virtualizado no son reproducibles entre sesiones. Se registró un factor de diez en el mismo binario sin recompilar. La metodología de medición intercalada mitiga el problema para las comparaciones relativas, pero ninguna cifra absoluta de tiempo del presente documento debe interpretarse como característica del sistema.

El corte por falta de progreso introduce un parámetro arbitrario. Su valor determina cuánto espera un hilo antes de rendirse, y aunque no afecta a la corrupción medida ni interviene en las variantes sincronizadas, sí influye en cuántos ítems alcanza a consumir la versión A antes de detenerse. Un valor distinto produciría porcentajes de entrega ligeramente distintos.

La carga de trabajo es sintética y homogénea. Todos los productores generan ítems a la máxima velocidad posible y sin cómputo asociado. Una carga con tiempos de producción

variables o con trabajo dentro de la sección crítica alteraría el perfil de contención y probablemente desplazaría el punto de cruce entre estrategias de espera.

No se midió la equidad entre hilos. El diseño no registra cuántos ítems procesó cada hilo individualmente, de modo que no puede descartarse que alguna variante favorezca sistemáticamente a determinados hilos ni evaluarse la propiedad de espera acotada enunciada en la Sección 3.2.

La verificación con ThreadSanitizer se realizó con carga reducida. Se emplearon 2 000 ítems en lugar de 50 000 debido a la penalización de rendimiento del analizador. La ausencia de hallazgos en las variantes sincronizadas es evidencia sólida pero no constituye una prueba de corrección para toda ejecución posible.

9. Conclusiones

El proyecto produjo un simulador del problema Productor-Consumidor con cuatro implementaciones intercambiables del buffer compartido, acompañado de una infraestructura de medición cuya validez descansa en que ningún hilo comparte estructuras de instrumentación durante la ejecución. Esa decisión de diseño es la que permite afirmar que el fenómeno medido es no determinista mientras la medición es determinista y auditable.

El resultado central en materia de corrección es inequívoco: la versión sin sincronización entrega correctamente menos del 40 % de los ítems que produce, con corrupción de los tres tipos previstos, mientras que las tres versiones sincronizadas entregan la totalidad sin una sola excepción en ninguna corrida. El costo medido de esa corrección, normalizado por ítem correctamente entregado, se sitúa en torno a un orden de magnitud bajo contención elevada.

Tres aprendizajes concretos se derivan del desarrollo. El primero es que la corrección en concurrencia no depende de la cantidad de código protector sino de la ubicación exacta de la frontera de la sección crítica: la diferencia entre una implementación que corrompe el 60 % de sus datos y una que no falla nunca es que la comprobación de la condición ocurra dentro o fuera del mismo bloqueo.

El segundo es que no existe un mecanismo de espera universalmente preferible. La espera activa y la espera bloqueante se alternan en conveniencia según la relación entre el tiempo esperado de espera y el costo de un cambio de contexto, mediada por la disponibilidad de procesadores. El experimento de control que reprodujo el mismo cruce en una plataforma distinta, sin más cambio que elevar el número de hilos, aísla esa variable de forma concluyente y explica de paso por qué un kernel como xv6 elige el spinlock para sus secciones críticas breves.

El tercero es de método más que de contenido, y surgió de un error propio: la verificación en la segunda plataforma no

confirmó una conclusión previa sino que la corrigió. A partir de los datos del primer entorno se había concluido que con buffer grande conviene bloquear, afirmación que resultó falsa en general al no reproducirse cuando había núcleos disponibles en abundancia. Sin la segunda plataforma esa conclusión errónea habría quedado incorporada al documento. El episodio ilustra que la verificación multiplataforma no es un requisito formal de portabilidad sino un instrumento de control experimental.

Como extensiones fuera del alcance del curso se identifican, en orden de valor: medir la equidad entre hilos para evaluar empíricamente la propiedad de espera acotada; incorporar el problema de Lectores-Escritores para contrastar un patrón de acceso con concurrencia de lectura; modelar tiempos de producción y consumo variables para desplazar el perfil de contención; e implementar el algoritmo del banquero como estrategia de evitación de interbloqueos, complementando las de prevención y detección aquí demostradas.

10. Referencias

- [1] A. Silberschatz, P. B. Galvin, y G. Gagne, Operating System Concepts, 10.^a ed. Hoboken, NJ, EE. UU.: Wiley, 2018.
- [2] A. S. Tanenbaum y H. Bos, Modern Operating Systems, 4.^a ed. Boston, MA, EE. UU.: Pearson, 2015.
- [3] R. H. Arpaci-Dusseau y A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces. Madison, WI, EE. UU.: Arpaci-Dusseau Books, 2018.
- [4] ISO/IEC, ISO/IEC 25010:2011 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models. Ginebra, Suiza: ISO, 2011.